

Node.js Backend Assignments

Detailed Report

Name	Stuti Saxena
Roll Number	024
Cohort	Larry Page
Technology	Node.js / JavaScript

1. Introduction

This report documents five Node.js assignments completed as a progression from a basic HTTP server to a multi-page personal portfolio. The assignments use Node.js and its built-in **http** module to practice server creation, requests and responses, HTML responses, JSON responses, routing, HTTP status codes, and multiple endpoints.

The work was developed incrementally: first creating a server, then returning HTML, then returning JSON from a specific API route, then adding multiple routes, and finally combining those concepts into a personal portfolio server.

2. Assignment 1 — Basic HTTP Server

Objective: Create a basic HTTP server using Node.js that displays "Welcome to Node.js Server" in the browser and runs on port 3000.

How it was solved:

The built-in **http** module was imported with **require('http')**. A server was created with **http.createServer()**. Inside the request handler, the response header was set to plain text and the required welcome message was sent with **res.end()**. Finally, **server.listen(3000)** was used to start the server on port 3000, with a terminal message confirming that it was running.

Expected result: Browser — Welcome to Node.js Server.

Concepts demonstrated: `http.createServer()`, server listening, request and response.

3. Assignment 2 — HTML Response Server

Objective: Serve an HTML page containing a Student Portal heading, student name, course, college name, and a welcome paragraph.

How it was solved:

The same HTTP server structure was extended so that the response content type was changed to **text/html**. An HTML document was returned from **res.end()**, including the Student Portal heading and the required student details. Horizontal rule elements (**<hr>**) were added above and below the heading to match the formatting shown in the expected output.

Page content: Student Portal; Name: John Doe; Course: Full Stack Development; College: XYZ College; Welcome to our Node.js application.

Concepts demonstrated: HTTP response, HTML content, and response headers.

4. Assignment 3 — Student JSON API

Objective: Create a simple REST-like endpoint where `/student` returns the specified student object as JSON, while other routes return 404 - Page Not Found.

How it was solved:

Route handling was introduced with an **if** condition checking `req.url === '/student'`. For that route, the response content type was set to **application/json**. A JavaScript object containing id 101, name John, course BCA, semester 4, and city Mumbai was converted to JSON using **JSON.stringify()** and returned to the browser. An **else** branch handled all other URLs with HTTP status 404 and the required message.

Student data: id 101; name John; course BCA; semester 4; city Mumbai.

Concepts demonstrated: JSON response, routing, and HTTP status codes.

5. Assignment 4 — Route Handling Server

Objective: Implement multiple routes using the Node.js HTTP module.

How it was solved:

The request URL was checked using an **if / else if / else** structure. Each valid route sends a 200 response and its required text. The final **else** branch handles any unknown route with a 404 response.

Route	Response
/	Welcome to Home Page
/about	About Us
/contact	Contact Information
/services	Our Services
Any other route	404 - Page Not Found

Concepts demonstrated: routing, URL handling, and HTTP status codes.

6. Assignment 5 — Personal Portfolio Server

Objective: Create a simple portfolio website using only the Node.js HTTP module, with separate pages for Home, About Me, Skills, Projects, and Contact Details. Every page must contain relevant HTML content and navigation, and invalid routes must show 404 - Page Not Found.

How it was solved:

The earlier routing approach was expanded into five portfolio routes. A reusable navigation section was created so users can move between all pages. A common HTML structure was also used for consistent page layout, including a header, navigation bar, main content area, and footer. Basic CSS was included inside the HTML to improve readability and presentation.

Route	Purpose
/	Home — introduction and personal information
/about	About Me — education, interests, and career mindset

Route	Purpose
/skills	Skills — programming, web, database, backend, and soft skills
/projects	Projects — Student Management System
/contact	Contact Details — email, LinkedIn, and GitHub

The portfolio content was personalized with Stuti Saxena's details: B.Tech Computer Science Engineering, ITM Skills University, School of Future Tech, skills including Python, C++, HTML5, CSS3, JavaScript, React JavaScript, Scratch, soft skills, Java, and learning areas including DBMS/SQL and Node.js backend. The project section includes a Student Management System built with HTML, CSS, and JavaScript. Contact information includes the supplied academic email, LinkedIn, and GitHub profiles.

Concepts demonstrated: routing, HTML responses, HTTP server, and multiple endpoints.

7. Technologies and Concepts Used

The assignments primarily use Node.js, JavaScript, the built-in HTTP module, HTML5, CSS3, and JSON. Across the five tasks, the implementation practices include `http.createServer()`, `server.listen()`, request URL checking, response headers, `res.end()`, `JSON.stringify()`, HTML generation, routing, and HTTP 404 status handling.

8. Testing and Execution

Each server was designed to run on port 3000. The programs can be started from the terminal with the Node.js command followed by the relevant JavaScript filename. The browser is then used to test the required URL or route. For assignments involving invalid routes, a non-existent path can be entered to verify the 404 response.

During development of Assignment 4, a duplicate response-header issue was encountered when a 200 header was written before the route checks and a 404 header was later written for an invalid route. This was resolved by moving the `res.writeHead()` call into each route branch, ensuring that only one response header is sent for each request.

9. Learning Outcomes

- Create and run a Node.js HTTP server.
- Handle browser requests and server responses.
- Return plain-text, HTML, and JSON responses.
- Implement single and multiple URL routes.
- Use HTTP 200 and 404 status codes appropriately.
- Set response headers according to the response type.
- Use `JSON.stringify()` to return JavaScript data as JSON.
- Build a simple multi-page website using Node.js.
- Combine backend routing with HTML and CSS.
- Debug a response-header error and correct the server logic.

10. Conclusion

The five assignments provide a step-by-step introduction to backend development with Node.js. The work progresses from a basic HTTP server to HTML responses, a JSON API, multiple route handling, and finally a

personalized portfolio. Together, the assignments demonstrate the practical foundations of HTTP servers, routing, responses, status codes, and multiple endpoints.

Prepared by: **Stuti Saxena** | Roll No.: **024** | Cohort: **Larry Page**